

# Turing Interview Prep: Senior Software Engineer – Go (LLM Evaluation & Repository Validation)

---

Complete preparation guide for Golang-focused interview and coding rounds.

## Role Overview

### Focus Areas:

1. **Go/Golang Expertise** - Concurrency, channels, interfaces, performance
  2. **LLM Evaluation** - Metrics, benchmarking, quality assessment
  3. **Repository Validation** - Code analysis, static checking, CI/CD
  4. **System Design** - Scalable evaluation pipelines
- 

## Go/Golang Core Concepts

### 1. Goroutines and Concurrency

```
package main

import (
    "fmt"
    "sync"
    "time"
)

// Concurrent LLM evaluation
type EvaluationTask struct {
    ModelName string
    Prompt    string
    Response  string
}

type EvaluationResult struct {
    Task    EvaluationTask
    Score   float64
    Error   error
}

// Evaluate multiple prompts concurrently
func EvaluateConcurrently(tasks []EvaluationTask, workers int) []EvaluationResult {
    taskChan := make(chan EvaluationTask, len(tasks))
    resultChan := make(chan EvaluationResult, len(tasks))

    var wg sync.WaitGroup

    // Start workers
```

```

    for i := 0; i < workers; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for task := range taskChan {
                result := evaluateTask(task)
                resultChan <- result
            }
        }()
    }

    // Send tasks
    for _, task := range tasks {
        taskChan <- task
    }
    close(taskChan)

    // Wait and collect results
    go func() {
        wg.Wait()
        close(resultChan)
    }()

    results := make([]EvaluationResult, 0, len(tasks))
    for result := range resultChan {
        results = append(results, result)
    }

    return results
}

func evaluateTask(task EvaluationTask) EvaluationResult {
    // Simulate evaluation
    time.Sleep(100 * time.Millisecond)

    score := calculateBLEUScore(task.Response, "reference")

    return EvaluationResult{
        Task: task,
        Score: score,
        Error: nil,
    }
}

// Worker pool pattern
type WorkerPool struct {
    maxWorkers int
    taskQueue  chan EvaluationTask
    results    chan EvaluationResult
    wg         sync.WaitGroup
}

func NewWorkerPool(maxWorkers, queueSize int) *WorkerPool {
    return &WorkerPool{

```

```

        maxWorkers: maxWorkers,
        taskQueue: make(chan EvaluationTask, queueSize),
        results:    make(chan EvaluationResult, queueSize),
    }
}

func (p *WorkerPool) Start() {
    for i := 0; i < p.maxWorkers; i++ {
        p.wg.Add(1)
        go p.worker()
    }
}

func (p *WorkerPool) worker() {
    defer p.wg.Done()
    for task := range p.taskQueue {
        result := evaluateTask(task)
        p.results <- result
    }
}

func (p *WorkerPool) Submit(task EvaluationTask) {
    p.taskQueue <- task
}

func (p *WorkerPool) Stop() {
    close(p.taskQueue)
    p.wg.Wait()
    close(p.results)
}

func (p *WorkerPool) Results() <-chan EvaluationResult {
    return p.results
}

```

## 2. Channels and Select

```

// Timeout pattern
func EvaluateWithTimeout(task EvaluationTask, timeout time.Duration)
(EvaluationResult, error) {
    resultChan := make(chan EvaluationResult, 1)

    go func() {
        result := evaluateTask(task)
        resultChan <- result
    }()

    select {
    case result := <-resultChan:
        return result, nil
    case <-time.After(timeout):

```

```

        return EvaluationResult{}, fmt.Errorf("evaluation timeout")
    }
}

// Fan-out, fan-in pattern
func FanOutFanIn(tasks []EvaluationTask) []EvaluationResult {
    channels := make([]<-chan EvaluationResult, len(tasks))

    // Fan-out: Start goroutine for each task
    for i, task := range tasks {
        ch := make(chan EvaluationResult, 1)
        channels[i] = ch

        go func(t EvaluationTask, c <-chan EvaluationResult) {
            c <- evaluateTask(t)
        }(task, ch)
    }

    // Fan-in: Merge results
    return merge(channels...)
}

func merge(channels ...<-chan EvaluationResult) []EvaluationResult {
    var wg sync.WaitGroup
    out := make(chan EvaluationResult, len(channels))

    wg.Add(len(channels))
    for _, ch := range channels {
        go func(c <-chan EvaluationResult) {
            defer wg.Done()
            for result := range c {
                out <- result
            }
        }(ch)
    }

    go func() {
        wg.Wait()
        close(out)
    }()

    results := make([]EvaluationResult, 0)
    for result := range out {
        results = append(results, result)
    }

    return results
}

// Pipeline pattern
func evaluationPipeline(tasks []EvaluationTask) <-chan EvaluationResult {
    // Stage 1: Generate
    taskStream := generate(tasks)

```

```

    // Stage 2: Evaluate
    evaluationStream := evaluate(taskStream)

    // Stage 3: Validate
    validationStream := validate(evaluationStream)

    return validationStream
}

func generate(tasks []EvaluationTask) <-chan EvaluationTask {
    out := make(chan EvaluationTask)
    go func() {
        defer close(out)
        for _, task := range tasks {
            out <- task
        }
    }()
    return out
}

func evaluate(in <-chan EvaluationTask) <-chan EvaluationResult {
    out := make(chan EvaluationResult)
    go func() {
        defer close(out)
        for task := range in {
            out <- evaluateTask(task)
        }
    }()
    return out
}

func validate(in <-chan EvaluationResult) <-chan EvaluationResult {
    out := make(chan EvaluationResult)
    go func() {
        defer close(out)
        for result := range in {
            // Validate score is within bounds
            if result.Score >= 0 && result.Score <= 1 {
                out <- result
            }
        }
    }()
    return out
}

```

### 3. Interfaces and Type System

```

// Evaluator interface
type Evaluator interface {
    Evaluate(response, reference string) (float64, error)
    Name() string
}

```

```
}

// BLEU evaluator
type BLEUEvaluator struct {
    NGram int
}

func (b *BLEUEvaluator) Evaluate(response, reference string) (float64, error) {
    score := calculateBLEUScore(response, reference)
    return score, nil
}

func (b *BLEUEvaluator) Name() string {
    return "BLEU"
}

// ROUGE evaluator
type ROUGEEvaluator struct {
    Variant string // "rouge-1", "rouge-l"
}

func (r *ROUGEEvaluator) Evaluate(response, reference string) (float64, error) {
    score := calculateROUGEScore(response, reference, r.Variant)
    return score, nil
}

func (r *ROUGEEvaluator) Name() string {
    return fmt.Sprintf("ROUGE-%s", r.Variant)
}

// Composite evaluator
type CompositeEvaluator struct {
    evaluators []Evaluator
}

func NewCompositeEvaluator(evaluators ...Evaluator) *CompositeEvaluator {
    return &CompositeEvaluator{evaluators: evaluators}
}

func (c *CompositeEvaluator) Evaluate(response, reference string)
(map[string]float64, error) {
    results := make(map[string]float64)

    for _, evaluator := range c.evaluators {
        score, err := evaluator.Evaluate(response, reference)
        if err != nil {
            return nil, fmt.Errorf("%s failed: %w", evaluator.Name(), err)
        }
        results[evaluator.Name()] = score
    }

    return results, nil
}
```

```
// Type assertion and type switch
func EvaluateWithType(evaluator interface{}, response, reference string) (float64, error) {
    // Type assertion
    if e, ok := evaluator.(Evaluator); ok {
        return e.Evaluate(response, reference)
    }

    // Type switch
    switch e := evaluator.(type) {
    case *BLEUEvaluator:
        return e.Evaluate(response, reference)
    case *ROUGEEvaluator:
        return e.Evaluate(response, reference)
    default:
        return 0, fmt.Errorf("unknown evaluator type")
    }
}
```

#### 4. Error Handling

```
import (
    "errors"
    "fmt"
)

// Custom errors
var (
    ErrInvalidScore   = errors.New("invalid score range")
    ErrEmptyResponse = errors.New("empty response")
    ErrTimeout        = errors.New("evaluation timeout")
)

// Error wrapping
type EvaluationError struct {
    Task EvaluationTask
    Err   error
}

func (e *EvaluationError) Error() string {
    return fmt.Sprintf("evaluation failed for model %s: %v", e.Task.ModelName, e.Err)
}

func (e *EvaluationError) Unwrap() error {
    return e.Err
}

// Error handling with defer
func SafeEvaluate(task EvaluationTask) (result EvaluationResult, err error) {
    defer func() {
```

```

        if r := recover(); r != nil {
            err = fmt.Errorf("panic during evaluation: %v", r)
        }
    }()

    if task.Response == "" {
        return EvaluationResult{}, ErrEmptyResponse
    }

    score := calculateBLEUScore(task.Response, "reference")

    if score < 0 || score > 1 {
        return EvaluationResult{}, fmt.Errorf("%w: got %f", ErrInvalidScore,
score)
    }

    return EvaluationResult{
        Task: task,
        Score: score,
        Error: nil,
    }, nil
}

// Error checking
func ProcessEvaluation(task EvaluationTask) error {
    result, err := SafeEvaluate(task)
    if err != nil {
        // Check for specific error
        if errors.Is(err, ErrEmptyResponse) {
            return fmt.Errorf("cannot evaluate empty response")
        }

        // Check if error is EvaluationError
        var evalErr *EvaluationError
        if errors.As(err, &evalErr) {
            fmt.Printf("Evaluation error for task: %v\n", evalErr.Task)
        }

        return err
    }

    fmt.Printf("Score: %.3f\n", result.Score)
    return nil
}

```

## 5. Context and Cancellation

```

import (
    "context"
    "time"
)

```



```
// Context-aware evaluation
func EvaluateWithContext(ctx context.Context, task EvaluationTask)
(EvaluationResult, error) {
    resultChan := make(chan EvaluationResult, 1)
    errChan := make(chan error, 1)

    go func() {
        result := evaluateTask(task)
        resultChan <- result
    }()

    select {
    case result := <-resultChan:
        return result, nil
    case err := <-errChan:
        return EvaluationResult{}, err
    case <-ctx.Done():
        return EvaluationResult{}, ctx.Err()
    }
}

// Batch evaluation with cancellation
func BatchEvaluateWithCancel(ctx context.Context, tasks []EvaluationTask)
([]EvaluationResult, error) {
    results := make([]EvaluationResult, 0, len(tasks))

    for _, task := range tasks {
        select {
        case <-ctx.Done():
            return results, ctx.Err()
        default:
            result, err := EvaluateWithContext(ctx, task)
            if err != nil {
                return results, err
            }
            results = append(results, result)
        }
    }

    return results, nil
}

// Usage with timeout
func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()

    tasks := []EvaluationTask{
        {ModelName: "gpt-4", Prompt: "test", Response: "response"},
    }

    results, err := BatchEvaluateWithCancel(ctx, tasks)
    if err != nil {
```

```
        if errors.Is(err, context.DeadlineExceeded) {
            fmt.Println("Evaluation timed out")
        }
    }
}
```

---

## LLM Evaluation in Go

### BLEU Score Implementation

```
package evaluation

import (
    "math"
    "strings"
)

// NGram generates n-grams from tokens
func NGram(tokens []string, n int) map[string]int {
    ngrams := make(map[string]int)

    for i := 0; i <= len(tokens)-n; i++ {
        ngram := strings.Join(tokens[i:i+n], " ")
        ngrams[ngram]++
    }

    return ngrams
}

// BLEUScore calculates BLEU score
func BLEUScore(candidate, reference string, maxN int) float64 {
    candTokens := strings.Fields(strings.ToLower(candidate))
    refTokens := strings.Fields(strings.ToLower(reference))

    if len(candTokens) == 0 {
        return 0
    }

    // Brevity penalty
    var bp float64
    candLen := float64(len(candTokens))
    refLen := float64(len(refTokens))

    if candLen < refLen {
        bp = math.Exp(1 - refLen/candLen)
    } else {
        bp = 1.0
    }

    // Calculate precision for each n-gram
```

```

precisions := make([]float64, 0, maxN)

for n := 1; n <= maxN; n++ {
    candNgrams := NGram(candTokens, n)
    refNgrams := NGram(refTokens, n)

    matches := 0
    total := 0

    for ngram, count := range candNgrams {
        total += count
        if refCount, exists := refNgrams[ngram]; exists {
            matches += min(count, refCount)
        }
    }

    if total == 0 {
        precisions = append(precisions, 0)
    } else {
        precisions = append(precisions, float64(matches)/float64(total))
    }
}

// Geometric mean
if minValue(precisions) == 0 {
    return 0
}

logSum := 0.0
for _, p := range precisions {
    logSum += math.Log(p)
}

geoMean := math.Exp(logSum / float64(len(precisions)))

return bp * geoMean
}

func min(a, b int) int {
    if a < b {
        return a
    }
    return b
}

func minValue(arr []float64) float64 {
    if len(arr) == 0 {
        return 0
    }

    minVal := arr[0]
    for _, v := range arr[1:] {
        if v < minVal {
            minVal = v
        }
    }
}

```

```

    }
}
return minVal
}

```

## Repository Validator in Go

```

package validator

import (
    "fmt"
    "go/ast"
    "go/parser"
    "go/token"
    "os"
    "path/filepath"
    "strings"
)

type ValidationResult struct {
    FilePath string
    Errors    []string
    Warnings []string
}

type RepositoryValidator struct {
    rootPath string
    results  []ValidationResult
}

func NewValidator(rootPath string) *RepositoryValidator {
    return &RepositoryValidator{
        rootPath: rootPath,
        results:  make([]ValidationResult, 0),
    }
}

// ValidateStructure checks repository structure
func (v *RepositoryValidator) ValidateStructure() error {
    required := []string{
        "README.md",
        "go.mod",
        ".gitignore",
    }

    for _, file := range required {
        path := filepath.Join(v.rootPath, file)
        if _, err := os.Stat(path); os.IsNotExist(err) {
            v.addError("", fmt.Sprintf("Missing required file: %s", file))
        }
    }
}

```

```

        return nil
    }

    // ValidateSyntax checks Go syntax
    func (v *RepositoryValidator) ValidateSyntax() error {
        return filepath.Walk(v.rootPath, func(path string, info os.FileInfo, err
error) error {
            if err != nil {
                return err
            }

            if !info.IsDir() && strings.HasSuffix(path, ".go") {
                if err := v.validateGoFile(path); err != nil {
                    v.addError(path, err.Error())
                }
            }

            return nil
        })
    }

    func (v *RepositoryValidator) validateGoFile(path string) error {
        fset := token.NewFileSet()

        file, err := parser.ParseFile(fset, path, nil, parser.AllErrors)
        if err != nil {
            return fmt.Errorf("syntax error: %w", err)
        }

        // Check for exported functions without documentation
        ast.Inspect(file, func(n ast.Node) bool {
            if fn, ok := n.(*ast.FuncDecl); ok {
                if fn.Name.IsExported() && fn.Doc == nil {
                    v.addWarning(path, fmt.Sprintf("Exported function %s lacks
documentation", fn.Name.Name))
                }
            }
            return true
        })

        return nil
    }

    // CheckComplexity analyzes code complexity
    func (v *RepositoryValidator) CheckComplexity(path string) int {
        fset := token.NewFileSet()

        file, err := parser.ParseFile(fset, path, nil, 0)
        if err != nil {
            return 0
        }

        complexity := 0

```

```
ast.Inspect(file, func(n ast.Node) bool {
    switch n.(type) {
    case *ast.IfStmt, *ast.ForStmt, *ast.RangeStmt,
        *ast.CaseClause, *ast.CommClause:
        complexity++
    }
    return true
})

return complexity
}

// GetFunctions extracts all functions from file
func (v *RepositoryValidator) GetFunctions(path string) ([]string, error) {
    fset := token.NewFileSet()

    file, err := parser.ParseFile(fset, path, nil, 0)
    if err != nil {
        return nil, err
    }

    functions := make([]string, 0)

    ast.Inspect(file, func(n ast.Node) bool {
        if fn, ok := n.(*ast.FuncDecl); ok {
            functions = append(functions, fn.Name.Name)
        }
        return true
    })

    return functions, nil
}

func (v *RepositoryValidator) addError(path, message string) {
    v.results = append(v.results, ValidationResult{
        FilePath: path,
        Errors:   []string{message},
    })
}

func (v *RepositoryValidator) addWarning(path, message string) {
    // Find existing result or create new
    for i := range v.results {
        if v.results[i].FilePath == path {
            v.results[i].Warnings = append(v.results[i].Warnings, message)
            return
        }
    }

    v.results = append(v.results, ValidationResult{
        FilePath: path,
        Warnings: []string{message},
    })
}
```

```
}

func (v *RepositoryValidator) GetResults() []ValidationResult {
    return v.results
}

// Generate report
func (v *RepositoryValidator) GenerateReport() string {
    var report strings.Builder

    report.WriteString("=== Repository Validation Report ===\n\n")

    totalErrors := 0
    totalWarnings := 0

    for _, result := range v.results {
        if len(result.Errors) > 0 || len(result.Warnings) > 0 {
            report.WriteString(fmt.Sprintf("File: %s\n", result.FilePath))

            for _, err := range result.Errors {
                report.WriteString(fmt.Sprintf("  ERROR: %s\n", err))
                totalErrors++
            }

            for _, warn := range result.Warnings {
                report.WriteString(fmt.Sprintf("  WARN: %s\n", warn))
                totalWarnings++
            }

            report.WriteString("\n")
        }
    }

    report.WriteString(fmt.Sprintf("Total Errors: %d\n", totalErrors))
    report.WriteString(fmt.Sprintf("Total Warnings: %d\n", totalWarnings))

    return report.String()
}
```

---

## Go-Specific Interview Questions

### Concurrency

#### Q1: What is the difference between goroutines and threads?

- Goroutines are lightweight (2KB initial stack vs 1-2MB for threads)
- Go runtime multiplexes goroutines onto OS threads (M:N scheduling)
- Goroutines are managed by Go scheduler, not OS
- Channel-based communication is idiomatic in Go

#### Q2: Explain the difference between buffered and unbuffered channels

- Unbuffered: `ch := make(chan int)` - sender blocks until receiver ready
- Buffered: `ch := make(chan int, 10)` - sender blocks only when buffer full
- Unbuffered provides synchronization guarantee
- Buffered can improve throughput but increases memory usage

**Q3: How do you prevent goroutine leaks?**

- Always ensure goroutines have a way to exit
- Use `context.Context` for cancellation
- Close channels when done sending
- Use `sync.WaitGroup` to wait for completion
- Avoid starting goroutines without cleanup mechanism

**Q4: What is the select statement used for?**

- Multiplexing multiple channel operations
- Non-blocking channel operations with `default` case
- Implementing timeouts with `time.After`
- Coordinating multiple concurrent operations

**Q5: Explain happens-before guarantee in Go**

- Memory model defines when reads/writes are visible across goroutines
- Channel send happens-before corresponding receive
- Closing channel happens-before receive of zero value
- `WaitGroup.Done()` happens-before `Wait()` returns

## Memory and Performance

**Q6: How does Go garbage collector work?**

- Concurrent mark-and-sweep collector
- Three-color marking (white, gray, black)
- Write barriers during marking phase
- STW (stop-the-world) pauses minimized
- Tunable with GOGC environment variable

**Q7: What causes memory leaks in Go?**

- Goroutine leaks (most common)
- Holding references to large objects
- Growing slices without bounds
- Maps that never shrink
- Unclosed resources (files, network connections)

**Q8: How do you optimize Go performance?**

- Profile with pprof (CPU, memory, goroutines)
- Use benchmarks: `go test -bench=.`
- Reduce allocations (use `sync.Pool`)
- Avoid reflection in hot paths



- Use buffered I/O
- Batch operations

### Q9: Explain slice internals and capacity vs length

```
// Slice: pointer, len, cap
s := make([]int, 5, 10) // len=5, cap=10
// Append beyond cap triggers reallocation
```

### Q10: What is escape analysis?

- Compiler determines if variable can stay on stack or must go to heap
- Stack allocation is much faster
- Use `go build -gcflags="-m"` to see escape analysis
- Large objects, closures, interface assignments often escape

## Interfaces and Type System

### Q11: What is the empty interface and when to use it?

```
var i interface{} // Can hold any value
// Use for: generic containers, unmarshaling JSON, reflection
// Avoid: prefer generics (Go 1.18+) or concrete types
```

### Q12: Explain interface embedding

```
type Reader interface { Read(p []byte) (n int, err error) }
type Writer interface { Write(p []byte) (n int, err error) }
type ReadWriter interface {
    Reader // Embedded
    Writer
}
```

### Q13: What is the difference between value and pointer receivers?

- Value receiver: `func (e Evaluator) Name() string` - receives copy
- Pointer receiver: `func (e *Evaluator) SetName(name string)` - can modify
- Use pointer for: modifications, large structs, consistency
- Interface with pointer methods requires pointer type

## Error Handling

### Q14: What are the best practices for error handling in Go?

- Return errors as last return value
- Check errors immediately

- Add context with `fmt.Errorf("context: %w", err)`
- Use `errors.Is` and `errors.As` for comparison
- Create sentinel errors: `var ErrNotFound = errors.New("not found")`
- Use custom error types for rich errors

### Q15: Explain error wrapping (Go 1.13+)

```
// Wrapping
err := fmt.Errorf("failed to evaluate: %w", originalErr)

// Checking
if errors.Is(err, ErrNotFound) { }

// Type assertion
var evalErr *EvaluationError
if errors.As(err, &evalErr) { }
```

---

## Coding Problems

### Problem 1: Concurrent LLM Batch Evaluator

**Task:** Implement a concurrent batch evaluator that:

- Evaluates 1000 prompts across 5 models
- Uses worker pool (max 10 concurrent evaluations)
- Has 5-second timeout per evaluation
- Returns aggregated results with error handling

```
package main

import (
    "context"
    "fmt"
    "sync"
    "time"
)

type Prompt struct {
    ID    int
    Text  string
}

type Evaluation struct {
    PromptID int
    Model    string
    Score    float64
    Duration time.Duration
    Error    error
}
```

```

}

type BatchEvaluator struct {
    maxWorkers int
    timeout    time.Duration
    models     []string
}

func NewBatchEvaluator(maxWorkers int, timeout time.Duration, models []string)
*BatchEvaluator {
    return &BatchEvaluator{
        maxWorkers: maxWorkers,
        timeout:    timeout,
        models:     models,
    }
}

func (b *BatchEvaluator) EvaluateBatch(ctx context.Context, prompts []Prompt)
[]Evaluation {
    type task struct {
        prompt Prompt
        model  string
    }

    // Create task queue
    tasks := make(chan task, len(prompts)*len(b.models))
    results := make(chan Evaluation, len(prompts)*len(b.models))

    // Generate all tasks
    for _, prompt := range prompts {
        for _, model := range b.models {
            tasks <- task{prompt: prompt, model: model}
        }
    }
    close(tasks)

    // Start workers
    var wg sync.WaitGroup
    for i := 0; i < b.maxWorkers; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for t := range tasks {
                select {
                case <-ctx.Done():
                    return
                default:
                    result := b.evaluateOne(ctx, t.prompt, t.model)
                    results <- result
                }
            }
        }()
    }
}

```

```

// Close results when done
go func() {
    wg.Wait()
    close(results)
}()

// Collect results
evaluations := make([]Evaluation, 0, len(prompts)*len(b.models))
for result := range results {
    evaluations = append(evaluations, result)
}

return evaluations
}

func (b *BatchEvaluator) evaluateOne(ctx context.Context, prompt Prompt, model
string) Evaluation {
    ctx, cancel := context.WithTimeout(ctx, b.timeout)
    defer cancel()

    start := time.Now()
    resultChan := make(chan float64, 1)
    errChan := make(chan error, 1)

    go func() {
        // Simulate LLM evaluation
        time.Sleep(time.Duration(100+prompt.ID%900) * time.Millisecond)
        score := float64(prompt.ID%100) / 100.0
        resultChan <- score
    }()

    select {
    case score := <-resultChan:
        return Evaluation{
            PromptID: prompt.ID,
            Model:     model,
            Score:     score,
            Duration: time.Since(start),
            Error:     nil,
        }
    case err := <-errChan:
        return Evaluation{
            PromptID: prompt.ID,
            Model:     model,
            Error:     err,
            Duration: time.Since(start),
        }
    case <-ctx.Done():
        return Evaluation{
            PromptID: prompt.ID,
            Model:     model,
            Error:     fmt.Errorf("timeout after %v", b.timeout),
            Duration: time.Since(start),
        }
    }
}

```

```

    }
}

// Aggregate results
func AggregateResults(evaluations []Evaluation) map[string]float64 {
    modelScores := make(map[string][]float64)

    for _, eval := range evaluations {
        if eval.Error == nil {
            modelScores[eval.Model] = append(modelScores[eval.Model], eval.Score)
        }
    }

    averages := make(map[string]float64)
    for model, scores := range modelScores {
        sum := 0.0
        for _, score := range scores {
            sum += score
        }
        averages[model] = sum / float64(len(scores))
    }

    return averages
}

func main() {
    // Create prompts
    prompts := make([]Prompt, 1000)
    for i := range prompts {
        prompts[i] = Prompt{ID: i, Text: fmt.Sprintf("prompt_%d", i)}
    }

    // Create evaluator
    models := []string{"gpt-4", "claude-3", "gemini-pro", "llama-3", "mixtral"}
    evaluator := NewBatchEvaluator(10, 5*time.Second, models)

    // Run evaluation
    ctx, cancel := context.WithTimeout(context.Background(), 2*time.Minute)
    defer cancel()

    start := time.Now()
    results := evaluator.EvaluateBatch(ctx, prompts)
    duration := time.Since(start)

    // Aggregate
    averages := AggregateResults(results)

    // Report
    fmt.Printf("Evaluated %d results in %v\n", len(results), duration)
    for model, avg := range averages {
        fmt.Printf("%s: %.3f\n", model, avg)
    }

    // Count errors

```

```
    errors := 0
    for _, r := range results {
        if r.Error != nil {
            errors++
        }
    }
    fmt.Printf("Errors: %d\n", errors)
}
```

## Problem 2: Repository AST Analyzer

**Task:** Build tool that analyzes Go repository and returns:

- All exported functions with complexity scores
- Files with high complexity (>20)
- Functions without documentation
- Unused imports

```
package main

import (
    "fmt"
    "go/ast"
    "go/parser"
    "go/token"
    "os"
    "path/filepath"
    "sort"
    "strings"
)

type FunctionInfo struct {
    Name      string
    File      string
    Complexity int
    Exported  bool
    HasDoc    bool
    Lines     int
}

type AnalysisReport struct {
    Functions      []FunctionInfo
    HighComplexity []string
    MissingDocs    []FunctionInfo
    TotalFiles     int
    TotalFunctions int
}

type Analyzer struct {
    rootPath string
    report   AnalysisReport
}
```

```

}

func NewAnalyzer(rootPath string) *Analyzer {
    return &Analyzer{
        rootPath: rootPath,
        report:   AnalysisReport{},
    }
}

func (a *Analyzer) Analyze() error {
    return filepath.Walk(a.rootPath, func(path string, info os.FileInfo, err
error) error {
        if err != nil {
            return err
        }

        // Skip vendor, .git, etc.
        if info.IsDir() {
            name := info.Name()
            if name == "vendor" || name == ".git" || name == "node_modules" {
                return filepath.SkipDir
            }
            return nil
        }

        // Only process .go files (not _test.go)
        if strings.HasSuffix(path, ".go") && !strings.HasSuffix(path, "_test.go")
{
            if err := a.analyzeFile(path); err != nil {
                fmt.Printf("Error analyzing %s: %v\n", path, err)
            }
        }

        return nil
    })
}

func (a *Analyzer) analyzeFile(path string) error {
    fset := token.NewFileSet()

    file, err := parser.ParseFile(fset, path, nil, parser.ParseComments)
    if err != nil {
        return err
    }

    a.report.TotalFiles++

    // Analyze functions
    ast.Inspect(file, func(n ast.Node) bool {
        if fn, ok := n.(*ast.FuncDecl); ok {
            info := a.analyzeFunctionComplexity(fn, path, fset)
            a.report.Functions = append(a.report.Functions, info)
            a.report.TotalFunctions++
        }
    })
}

```

```

        if info.Complexity > 20 {
            a.report.HighComplexity = append(a.report.HighComplexity,
                fmt.Sprintf("%s:%s (complexity: %d)", path, info.Name,
info.Complexity))
        }

        if info.Exported && !info.HasDoc {
            a.report.MissingDocs = append(a.report.MissingDocs, info)
        }
    }
    return true
})

return nil
}

func (a *Analyzer) analyzeFunctionComplexity(fn *ast.FuncDecl, file string, fset
*token.FileSet) FunctionInfo {
    complexity := 1 // Base complexity

    // Count decision points
    ast.Inspect(fn.Body, func(n ast.Node) bool {
        switch n.(type) {
        case *ast.IfStmt:
            complexity++
        case *ast.ForStmt, *ast.RangeStmt:
            complexity++
        case *ast.SwitchStmt, *ast.TypeSwitchStmt:
            complexity++
        case *ast.SelectStmt:
            complexity++
        case *ast.CaseClause:
            complexity++
        case *ast.BinaryExpr:
            // Count logical operators (&&, ||)
            if b, ok := n.(*ast.BinaryExpr); ok {
                if b.Op == token.LAND || b.Op == token.LOR {
                    complexity++
                }
            }
        }
    })
    return true
})

// Calculate lines
start := fset.Position(fn.Pos()).Line
end := fset.Position(fn.End()).Line
lines := end - start + 1

return FunctionInfo{
    Name:      fn.Name.Name,
    File:      file,
    Complexity: complexity,
    Exported:  fn.Name.IsExported(),
}

```



```

        HasDoc:      fn.Doc != nil && len(fn.Doc.List) > 0,
        Lines:      lines,
    }
}

func (a *Analyzer) GenerateReport() string {
    var report strings.Builder

    report.WriteString("=== Go Repository Analysis Report ===\n\n")

    report.WriteString(fmt.Sprintf("Total Files: %d\n", a.report.TotalFiles))
    report.WriteString(fmt.Sprintf("Total Functions: %d\n\n",
a.report.TotalFunctions))

    // High complexity functions
    if len(a.report.HighComplexity) > 0 {
        report.WriteString(fmt.Sprintf("High Complexity Functions (%d):\n",
len(a.report.HighComplexity)))
        for _, fn := range a.report.HighComplexity {
            report.WriteString(fmt.Sprintf("  - %s\n", fn))
        }
        report.WriteString("\n")
    }

    // Missing documentation
    if len(a.report.MissingDocs) > 0 {
        report.WriteString(fmt.Sprintf("Exported Functions Without Documentation
(%d):\n", len(a.report.MissingDocs)))
        for _, fn := range a.report.MissingDocs {
            report.WriteString(fmt.Sprintf("  - %s in %s\n", fn.Name, fn.File))
        }
        report.WriteString("\n")
    }

    // Top 10 most complex functions
    sorted := make([]FunctionInfo, len(a.report.Functions))
    copy(sorted, a.report.Functions)
    sort.Slice(sorted, func(i, j int) bool {
        return sorted[i].Complexity > sorted[j].Complexity
    })

    report.WriteString("Top 10 Most Complex Functions:\n")
    for i := 0; i < 10 && i < len(sorted); i++ {
        fn := sorted[i]
        report.WriteString(fmt.Sprintf("  %d. %s (complexity: %d, lines: %d) -
%s\n",
            i+1, fn.Name, fn.Complexity, fn.Lines, fn.File))
    }

    return report.String()
}

func main() {
    if len(os.Args) < 2 {

```

```
        fmt.Println("Usage: analyzer <repository-path>")
        os.Exit(1)
    }

    analyzer := NewAnalyzer(os.Args[1])

    if err := analyzer.Analyze(); err != nil {
        fmt.Printf("Analysis failed: %v\n", err)
        os.Exit(1)
    }

    fmt.Println(analyzer.GenerateReport())
}
```

### Problem 3: Benchmark LLM Evaluation Performance

**Task:** Write benchmarks comparing different BLEU implementations

```
package evaluation

import (
    "strings"
    "testing"
)

// Simple BLEU implementation
func BLEUSimple(candidate, reference string) float64 {
    // Simple token-based comparison
    candTokens := strings.Fields(candidate)
    refTokens := strings.Fields(reference)

    matches := 0
    for _, ct := range candTokens {
        for _, rt := range refTokens {
            if ct == rt {
                matches++
                break
            }
        }
    }

    if len(candTokens) == 0 {
        return 0
    }

    return float64(matches) / float64(len(candTokens))
}

// Map-based BLEU (optimized)
func BLEUOptimized(candidate, reference string) float64 {
    candTokens := strings.Fields(candidate)
```

```

    refTokens := strings.Fields(reference)

    // Build map for O(1) lookup
    refMap := make(map[string]int)
    for _, token := range refTokens {
        refMap[token]++
    }

    matches := 0
    for _, token := range candTokens {
        if count, exists := refMap[token]; exists && count > 0 {
            matches++
            refMap[token]--
        }
    }

    if len(candTokens) == 0 {
        return 0
    }

    return float64(matches) / float64(len(candTokens))
}

var result float64

func BenchmarkBLEUSimple(b *testing.B) {
    candidate := "the cat sat on the mat"
    reference := "the cat is on the mat"

    var r float64
    for i := 0; i < b.N; i++ {
        r = BLEUSimple(candidate, reference)
    }
    result = r
}

func BenchmarkBLEUOptimized(b *testing.B) {
    candidate := "the cat sat on the mat"
    reference := "the cat is on the mat"

    var r float64
    for i := 0; i < b.N; i++ {
        r = BLEUOptimized(candidate, reference)
    }
    result = r
}

func BenchmarkBLEULongText(b *testing.B) {
    candidate := strings.Repeat("the quick brown fox jumps over the lazy dog ",
100)
    reference := strings.Repeat("the quick brown fox jumped over the lazy dog ",
100)

    var r float64

```

```
for i := 0; i < b.N; i++ {
    r = BLEUOptimized(candidate, reference)
}
result = r
}
```

// Run with: go test -bench=. -benchmem

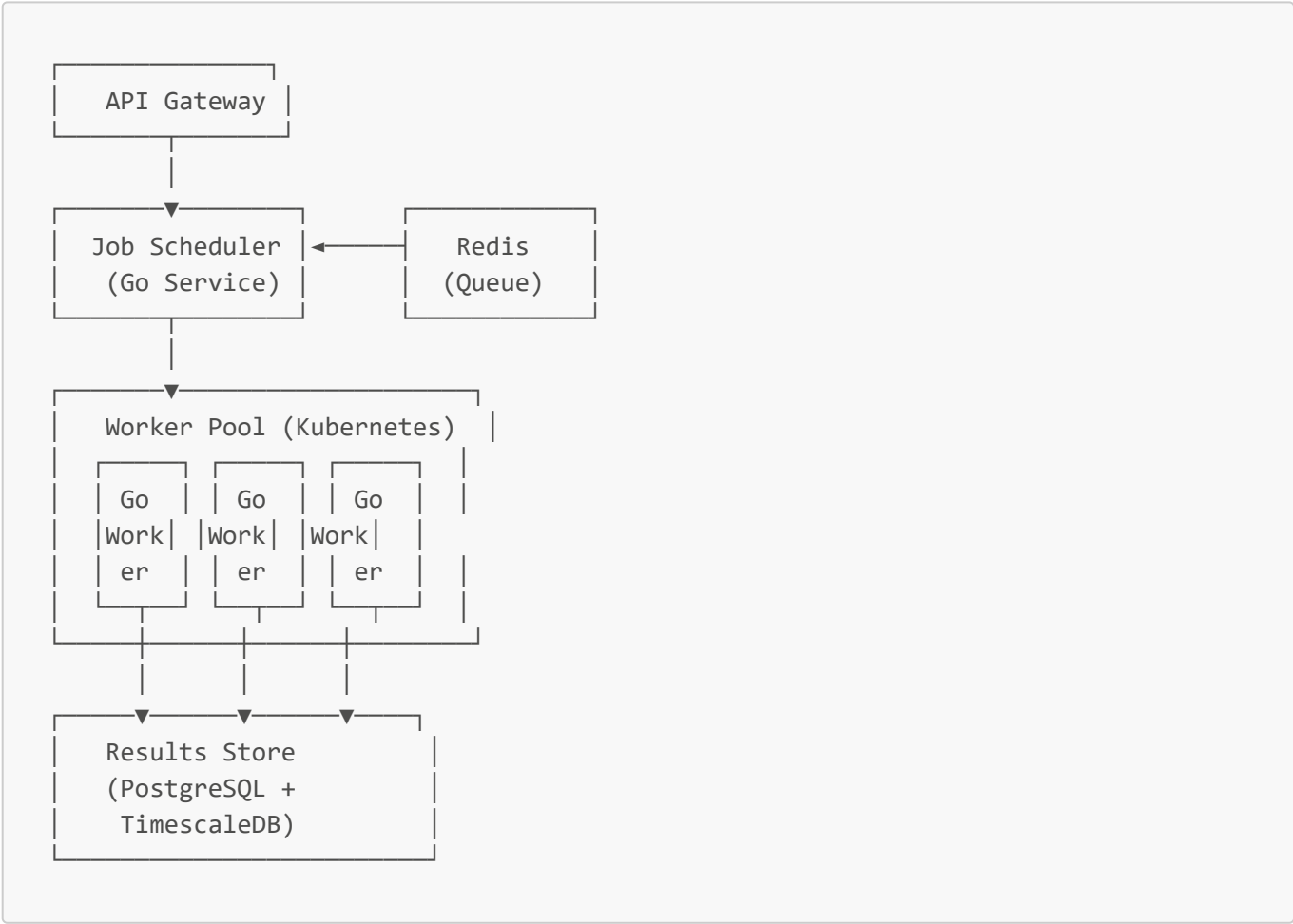
# System Design Questions

## Design 1: Distributed LLM Evaluation System

### Requirements:

- Evaluate 100K prompts daily across 10 models
- Support for custom evaluation metrics
- Real-time progress tracking
- Store results for historical comparison
- Handle model API rate limits

### Architecture:



### Key Components:

1. **API Gateway (Go + Gin)**

```

type EvaluationRequest struct {
    Prompts    []string    `json:"prompts"`
    Models     []string    `json:"models"`
    Metrics    []string    `json:"metrics"`
    Priority   int         `json:"priority"`
}

func SubmitJob(c *gin.Context) {
    var req EvaluationRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(400, gin.H{"error": err.Error()})
        return
    }

    jobID := generateJobID()

    // Push to Redis queue
    job := Job{
        ID:        jobID,
        Prompts:   req.Prompts,
        Models:    req.Models,
        Metrics:   req.Metrics,
        Priority:  req.Priority,
        Status:    "queued",
    }

    if err := queueJob(job); err != nil {
        c.JSON(500, gin.H{"error": "failed to queue job"})
        return
    }

    c.JSON(202, gin.H{"job_id": jobID})
}

```

## 2. Worker with Rate Limiting

```

type RateLimiter struct {
    limiter *rate.Limiter
    model  string
}

type Worker struct {
    id            int
    rateLimiters map[string]*RateLimiter
}

func (w *Worker) processJob(ctx context.Context, job Job) error {
    results := make([]EvaluationResult, 0)

    for _, model := range job.Models {

```

```

    limiter := w.rateLimiters[model]

    for _, prompt := range job.Prompts {
        // Wait for rate limit
        if err := limiter.limiter.Wait(ctx); err != nil {
            return err
        }

        result, err := w.evaluate(prompt, model, job.Metrics)
        if err != nil {
            // Retry with exponential backoff
            result, err = w.retryEvaluate(ctx, prompt, model, job.Metrics)
        }

        results = append(results, result)

        // Update progress
        w.updateProgress(job.ID, len(results),
            len(job.Prompts)*len(job.Models))
    }
}

// Store results
return w.storeResults(job.ID, results)
}

```

### 3. Progress Tracking (WebSocket)

```

func StreamProgress(c *gin.Context) {
    jobID := c.Param("job_id")

    upgrader := websocket.Upgrader{}
    conn, err := upgrader.Upgrade(c.Writer, c.Request, nil)
    if err != nil {
        return
    }
    defer conn.Close()

    ticker := time.NewTicker(1 * time.Second)
    defer ticker.Stop()

    for {
        select {
        case <-ticker.C:
            progress := getProgress(jobID)
            if err := conn.WriteJSON(progress); err != nil {
                return
            }

            if progress.Status == "completed" {
                return
            }
        }
    }
}

```

```
}  
}  
}
```

**Trade-offs:**

- Go for workers: Fast, low memory, excellent concurrency
- Redis for queue: Simple, fast, supports priority queues
- TimescaleDB: Time-series data for historical analysis
- Kubernetes: Auto-scaling based on queue depth

## Design 2: Real-Time Model Comparison Dashboard

**Requirements:**

- Compare 5 models side-by-side
- Display metrics: latency, accuracy, cost
- Live updates
- Historical trends

**Tech Stack:**

- Backend: Go + gRPC
- Frontend: Next.js + WebSocket
- Cache: Redis
- Metrics: Prometheus + Grafana
- Database: PostgreSQL

---

## Behavioral Questions

**Q1: Tell me about a time you optimized Go application performance**

Example answer structure:

- **Situation:** Service handling 100 req/s with 2s latency
- **Task:** Reduce latency to <500ms
- **Action:**
  - Profiled with pprof: found 60% time in JSON marshaling
  - Used sync.Pool for reusing buffers
  - Implemented connection pooling
  - Reduced allocations by 70%
- **Result:** Latency dropped to 300ms, handled 500 req/s

**Q2: Describe a complex concurrency bug you debugged****Q3: How do you ensure code quality in Go projects?**

- Code review process
- golangci-lint with strict rules

- Unit tests (80%+ coverage target)
- Integration tests with testcontainers
- Benchmarks for critical paths
- Race detector in CI: `go test -race`

**Q4: Explain a time you had to refactor legacy Go code**

**Q5: How do you handle disagreements in technical decisions?**

---

## Study Plan (2 Weeks)

Week 1: Go Fundamentals + LLM Basics

### Days 1-2: Concurrency Deep Dive

- Practice: Implement worker pool, fan-out/fan-in, pipeline
- Read: "Concurrency in Go" by Katherine Cox-Buday
- Exercise: Build concurrent web scraper

### Days 3-4: Performance & Profiling

- Learn pprof: CPU, memory, goroutine profiling
- Practice benchmarking
- Optimize sample code
- Exercise: Profile and optimize BLEU implementation

### Days 5-6: LLM Evaluation Metrics

- Implement BLEU, ROUGE from scratch
- Study BERTScore, Perplexity
- Practice with real datasets
- Exercise: Build evaluation CLI tool

### Day 7: Review & Practice

- Solve LeetCode hard problems in Go
- Review concurrency patterns
- Mock interview practice

Week 2: Advanced Topics + System Design

### Days 8-9: Repository Validation

- Study go/ast, go/parser packages
- Build AST analysis tools
- Practice static analysis
- Exercise: Build linter for custom rules

### Days 10-11: System Design

- Design distributed evaluation system



- Study microservices patterns in Go
- Practice capacity estimation
- Exercise: Design rate-limited API gateway

### Days 12-13: Advanced Go

- Generics (Go 1.18+)
- Context patterns
- Advanced error handling
- Testing strategies
- Exercise: Build generic evaluation framework

### Day 14: Final Review

- Review all coding problems
- Mock interviews
- System design practice
- Prepare questions for interviewer

---

## Resources

### Books

- **"The Go Programming Language"** by Donovan & Kernighan
- **"Concurrency in Go"** by Katherine Cox-Buday
- **"Learning Go"** by Jon Bodner

### Online

- **Go by Example:** <https://gobyexample.com/>
- **Effective Go:** [https://go.dev/doc/effective\\_go](https://go.dev/doc/effective_go)
- **Go Blog:** <https://go.dev/blog/>
- **Awesome Go:** <https://github.com/avelino/awesome-go>

### Practice

- **LeetCode Go track:** Focus on concurrency problems
- **Exercism Go track:** <https://exercism.org/tracks/go>
- **Go Playground:** <https://go.dev/play/>

### Tools

- **golangci-lint:** Comprehensive linting
- **pprof:** Profiling tool
- **go-critic:** Additional static analysis
- **gotests:** Generate table-driven tests

---

## Turing-Specific Tips

## 1. Function Calling Assessment

- Focus on understanding tool selection logic
- Practice constructing JSON payloads
- Study multi-step reasoning

## 2. Coding Round

- Write idiomatic Go (gofmt, golint)
- Add comments for exported functions
- Handle errors explicitly
- Use table-driven tests
- Consider edge cases

## 3. System Design

- Start with requirements clarification
- Draw architecture diagrams
- Discuss trade-offs explicitly
- Mention monitoring and observability
- Consider scalability from start

## 4. Communication

- Think out loud during coding
- Explain your approach before coding
- Ask clarifying questions
- Discuss alternative approaches

## 5. Common Pitfalls

- Goroutine leaks (always have exit path)
- Race conditions (use `-race` flag)
- Not closing channels
- Ignoring context cancellation
- Over-optimization without profiling

Good luck with your Turing interview!